

AI-Assisted Coding
Lab Assignment - 14.4

Name: A. Sushmitha

Roll No: 2303A51541

Batch: 13

Lab 14: Web Frontend Development – AI-Assisted HTML/CSS/JS Generation

Task 1: Responsive Homepage Layout for a Startup Website

Scenario

You are designing a basic homepage for a startup company that needs to look good on desktops, tablets, and mobile devices.

Expected Output

- A responsive web page where:
 - o Header, content, and footer are clearly separated
 - o Layout adapts correctly to mobile, tablet, and desktop views
 - o Code is clean and well-structured

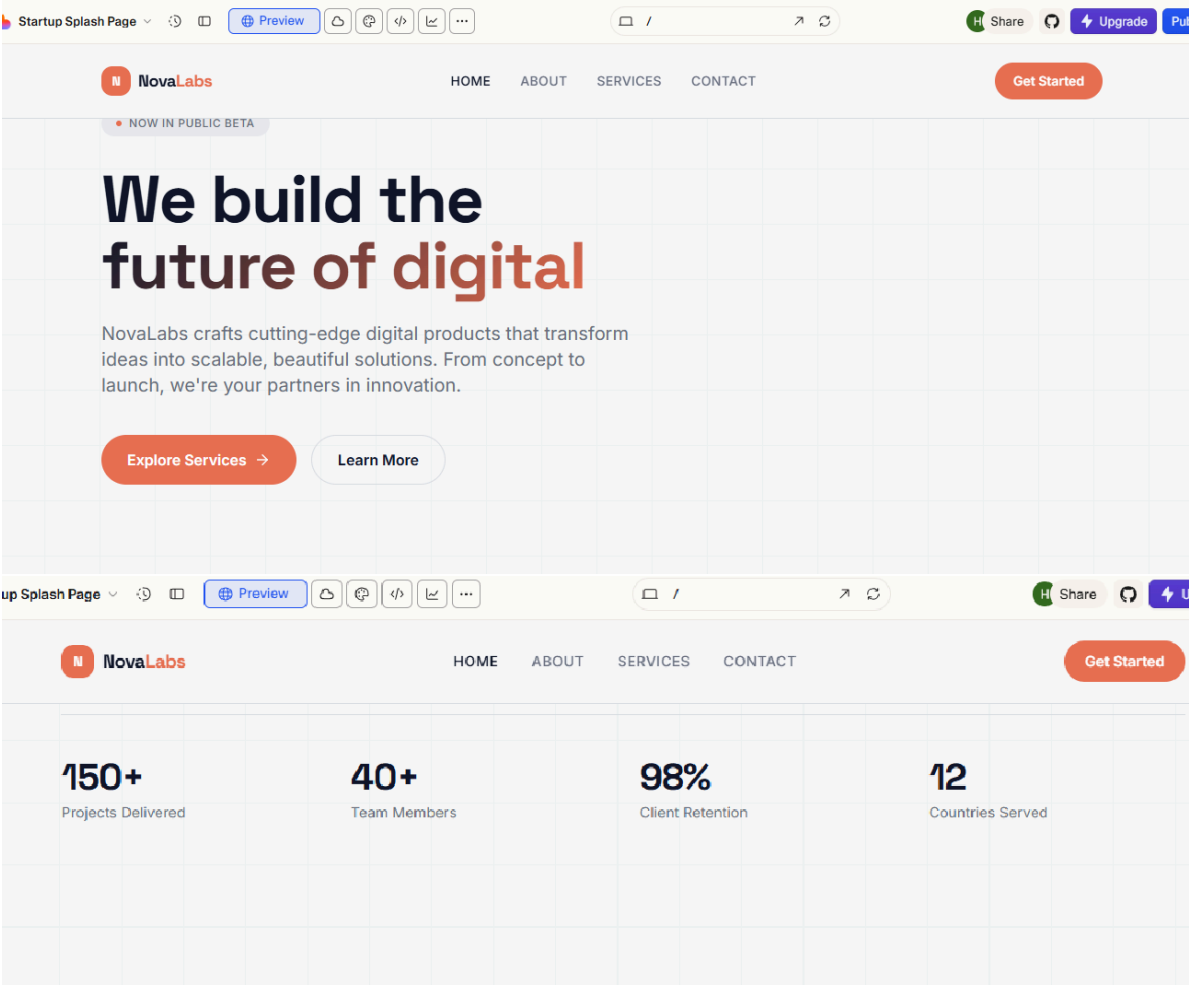
Prompt:

Generate a responsive homepage layout for a startup website using HTML and CSS.

Requirements:

- Include a header with a logo and navigation links (Home, About, Services, Contact)
- A main content section with a welcome message and description of the startup
- A footer with copyright information
- Use clean and semantic HTML structure
- Use CSS with media queries to make the layout responsive for desktop, tablet, and mobile devices
- Ensure navigation adjusts properly on smaller screens
- Write well-structured and readable code with comments

Output:



Why startups choose NovaLabs

We combine deep technical expertise with a design-first mindset to deliver products that users love and businesses rely on.



Lightning Fast

We ship products at breakneck speed without sacrificing quality or user experience.



Enterprise Security

Bank-grade security baked into every layer of your application from day one.



Scalable Architecture

Built to grow with you — from your first hundred users to your first million.

What we do best

End-to-end digital services tailored to ambitious startups and growing businesses.



Web Development

Full-stack applications built with modern frameworks and best practices.



UI/UX Design

Research-driven design that delights users and drives conversions.



Data & Analytics

Turn your data into actionable insights with custom dashboards and pipelines.



Mobile Apps

Cross-platform mobile experiences that feel native on every device.



Data & Analytics

Turn your data into actionable insights with custom dashboards and pipelines.



Mobile Apps

Cross-platform mobile experiences that feel native on every device.

Justification:

This task involves creating a responsive homepage layout for a startup website using HTML and CSS.

- HTML is used to create the structure of the webpage, which includes a header, main content section, and footer.
- The header contains the logo and navigation links like Home, About, Services, and Contact.
- The main section displays the main information or welcome message of the startup.
- The footer shows basic information such as copyright details.

To make the webpage work well on different devices (desktop, tablet, and mobile), CSS media queries are used. Media queries adjust the layout based on the screen size, ensuring the website looks organized and readable on all devices.

Task 2: Interactive Call-to-Action Button**Scenario**

A marketing page requires a call-to-action button that responds when users interact with it.

Expected Output

- A web page where:
 - o Clicking the button triggers a visible response (alert or message)
 - o JavaScript code is readable and commented
 - o No page reload occurs

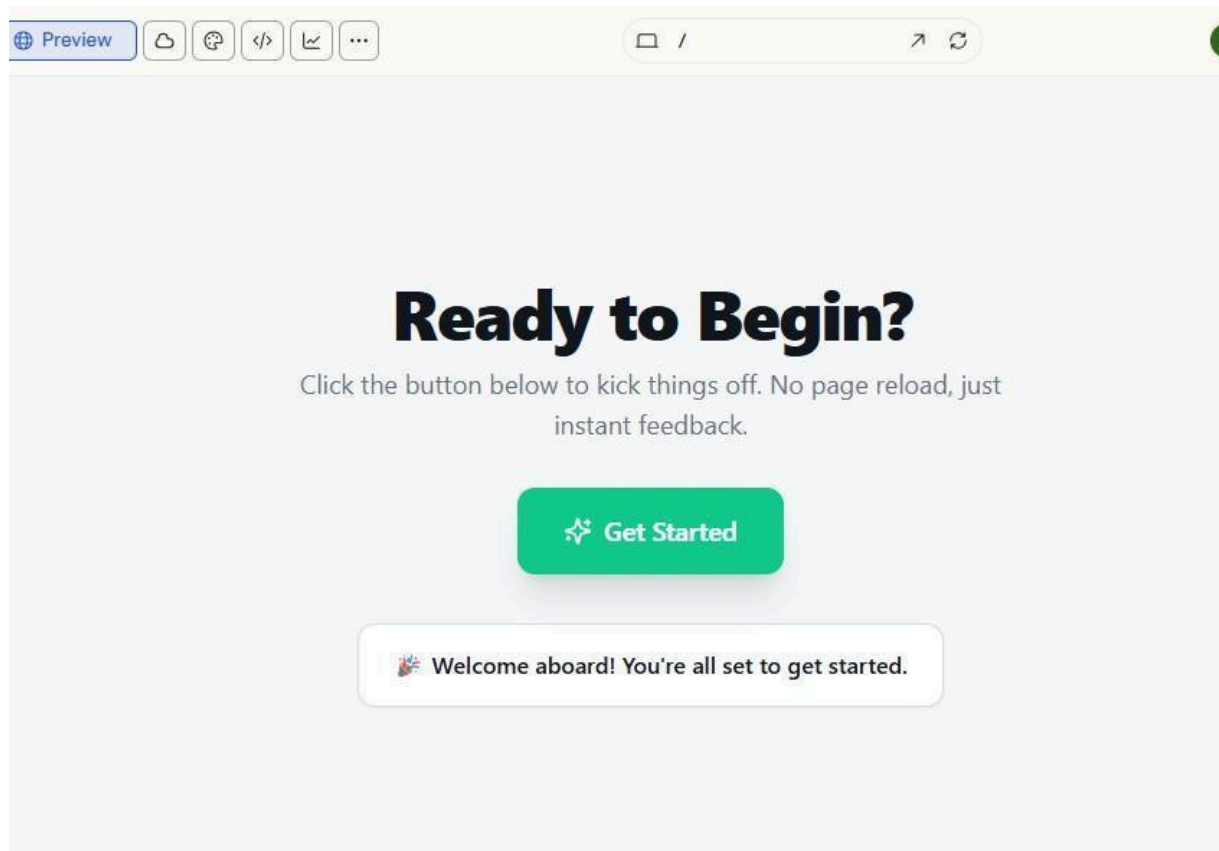
Prompt:

Create a simple web page with an interactive Call-to-Action

button. **Requirements:**

- Use HTML to add a button labeled "Get Started".
- Use JavaScript so that when the button is clicked, a message or alert appears.
- The page should not reload when the button is clicked.
- Write clean and readable JavaScript.
- Add meaningful comments explaining what each part of the JavaScript code does.

Output:



Justification:

In this task, an interactive Call-to-Action (CTA) button is added to a webpage using HTML. JavaScript is used to detect when the user clicks the button. When the button is clicked, a message or alert is displayed to the user. The JavaScript code includes comments to explain the logic, making it easier to understand. The interaction happens without reloading the page, providing a smooth user experience.

Task 3: Contact Form with Client-Side Validation

Scenario

You are developing a contact form for a company website to collect user queries safely and correctly.

Expected Output

- A functional form where:
 - o Invalid input shows inline error messages
 - o Valid input displays a success message
 - o Validation happens on the client side using JavaScript

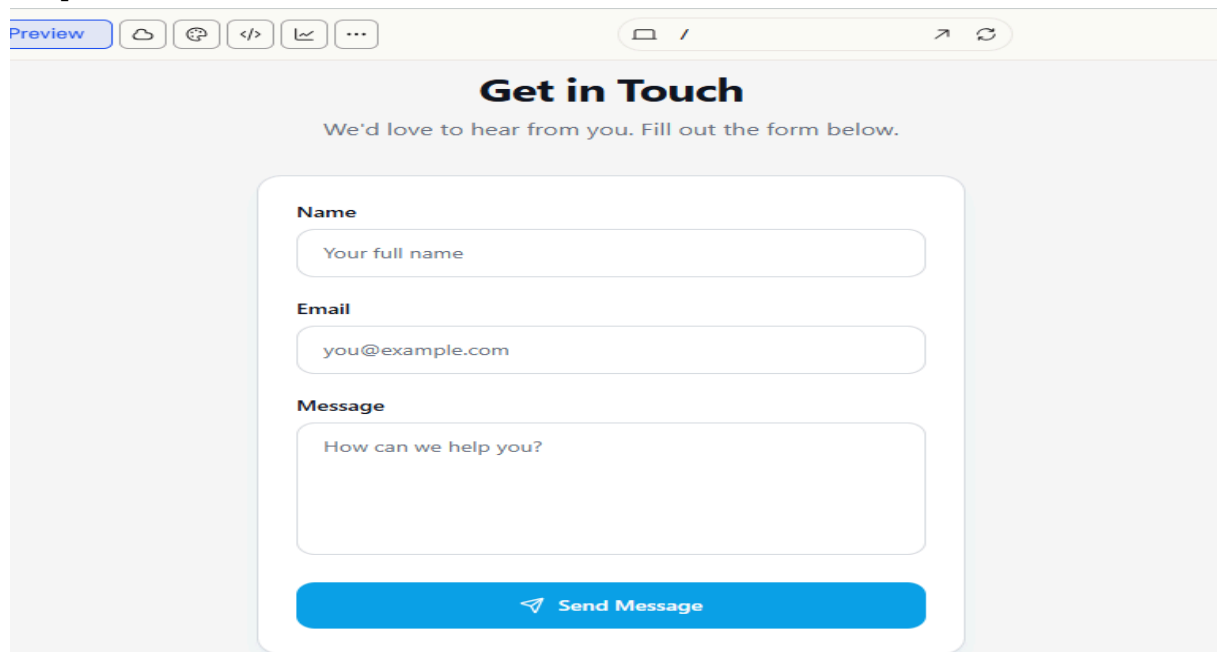
Prompt:

Create a contact form using HTML, CSS, and JavaScript.

Requirements:

- The form should include fields for Name, Email, and Message.
- Add a Submit button.
- Use JavaScript for client-side validation.
- Prevent the form from submitting if fields are empty.
- Validate the email format.
- Display clear inline error messages for invalid inputs.
- Show a success message when the form is filled correctly.
- Add comments explaining the JavaScript validation logic.

Output:



The image shows a web browser window with a 'Preview' button and various icons in the top bar. The main content is a contact form titled 'Get in Touch' with the subtitle 'We'd love to hear from you. Fill out the form below.' The form is centered and contains three input fields: 'Name' with placeholder text 'Your full name', 'Email' with placeholder text 'you@example.com', and 'Message' with placeholder text 'How can we help you?'. Below these fields is a blue 'Send Message' button with a paper plane icon.

Justification:

This task creates a contact form using HTML and JavaScript. The form includes fields for Name, Email, and Message. JavaScript validation checks the user input before submitting the form. It ensures that all fields are filled and the email is in a correct format. If the input is invalid, error messages are shown near the fields. If the input is correct, a success message is displayed. The validation happens on the client side without reloading the page.

Task 4: Dynamic List Management for a Product Page

Scenario

A product listing page needs to add or remove items dynamically without refreshing the page.

Expected Output

- A web page where:
 - o Items can be added and removed dynamically
 - o Changes appear instantly without page reload
 - o JavaScript logic is clear and maintainable

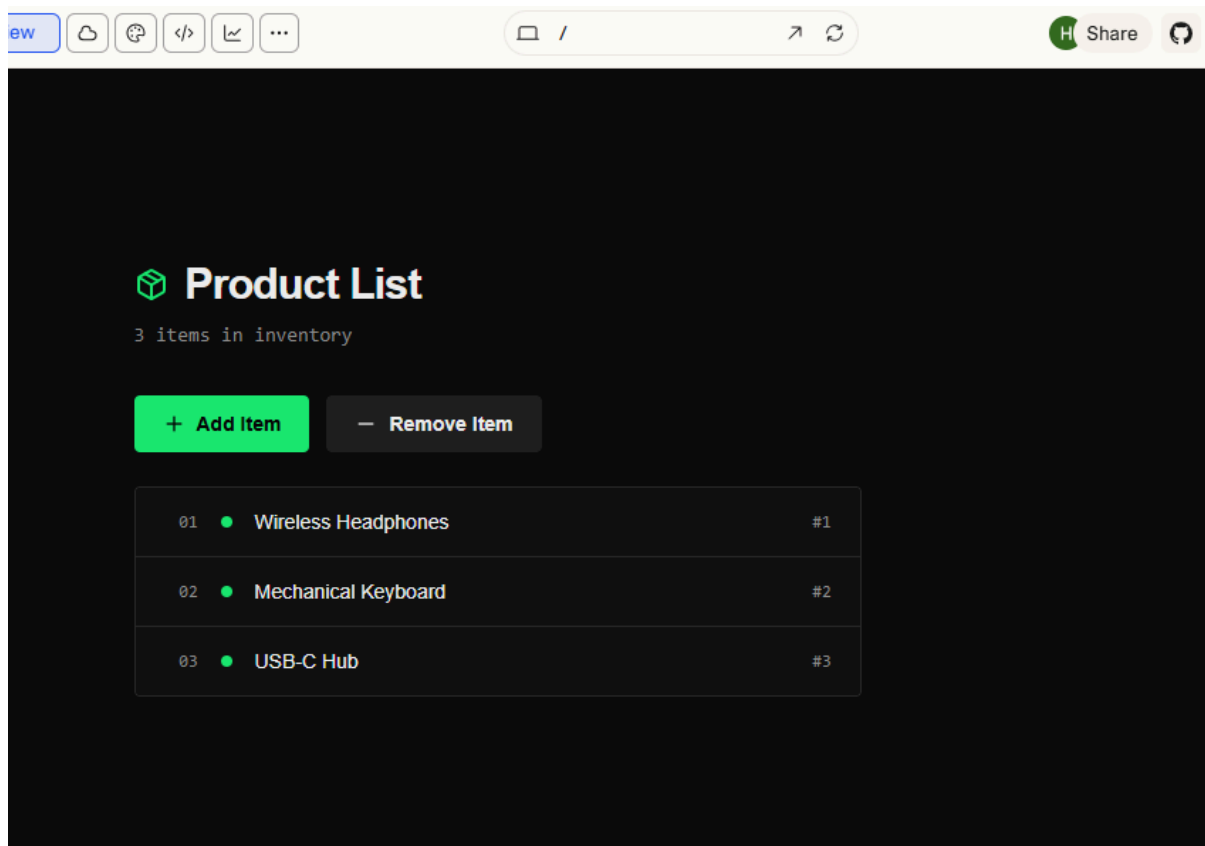
Prompt:

Create a web page that manages a dynamic product list using HTML and JavaScript.

Requirements:

- Use HTML to create a list that displays product items.
- Add two buttons: "Add Item" and "Remove Item".
- Use JavaScript to add a new item to the list when the Add button is clicked.
- Remove the last item from the list when the Remove button is clicked.
- Ensure the list updates dynamically without reloading the page.
- Write clean and well-commented JavaScript code.

Output:



Justification:

This task creates a dynamic product list using HTML and JavaScript. HTML is used to create a list of items and buttons for adding or removing products. JavaScript is used to modify the DOM when the buttons are clicked. When the Add button is clicked, a new item is added to the list. When the Remove button is clicked, the last item is removed. The changes appear instantly without refreshing the page, making the page interactive.

Task 5: Modal Popup for User Notifications**Scenario**

You are building a user notification popup for announcements or alerts on a website.

Expected Output

- A web page where:
 - o Clicking “Open Modal” displays the popup
 - o Popup overlays the page content clearly
 - o Modal closes smoothly when instructed

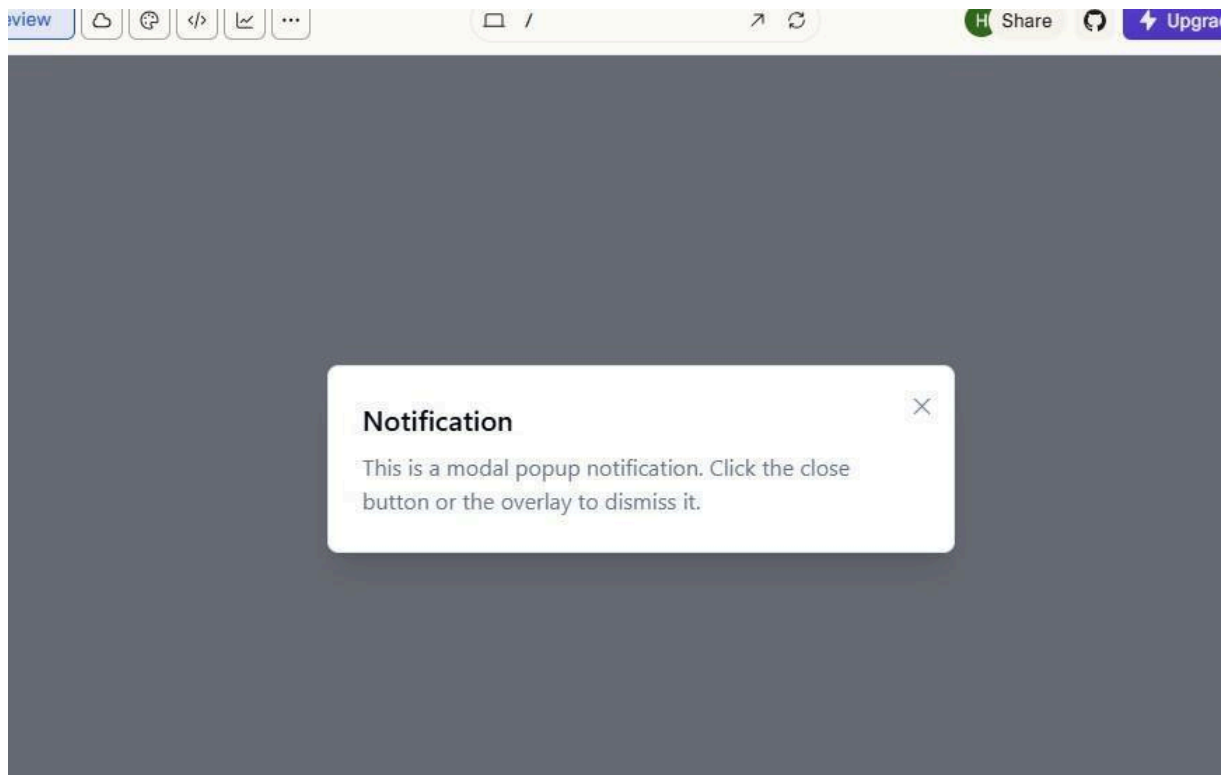
Prompt:

Create a modal popup notification using HTML, CSS, and

JavaScript. Requirements:

- Add a button labeled "Open Modal".
- Create a modal popup that appears in the center of the page.
- Add a semi-transparent background overlay behind the modal.
- Use JavaScript to open the modal when the button is clicked.
- Allow the modal to close when the user clicks the close button or the overlay.
- Write clean and well-commented code.

Output:



Justification:

This task creates a modal popup notification using HTML, CSS, and JavaScript. HTML is used to build the modal structure and the open button. CSS is used to design the popup and a semi-transparent background overlay that appears above the page content. JavaScript controls the opening and closing of the modal. The popup opens when the “Open Modal” button is clicked and can be closed by clicking the close button or the background overlay, creating a smooth user interaction.